ANNEXURE A

Analytical Problem Solving

Solutions to Assessment Questions 1 – 5

Q.	Topic
1	Water Jug Problem (state-space search)
2	Mars Rover – PEAS and agent architecture
3	8-Queens Problem – problem formulation and search
4	OLA Cab Booking – agent type and pseudocode
5	Logistics Network – Uniform Cost Search (UCS)

Question 1: Water Jug Problem

Question: *Using a 4-gallon jug and a 3-gallon jug (no markings) and a pump, measure exactly 2 gallons of water into the 4-gallon jug.*

Problem formulation

- **State:** (x, y) where x = water in the 4-gallon jug (0–4) and y = water in the 3-gallon jug (0–3).
- **Initial state:** (0, 0)
- **Goal state:** (2, y) for any y, i.e. the 4-gallon jug holds exactly 2 gallons.
- **Actions:** Fill a jug from the pump; empty a jug onto the ground; pour one jug into the other until the target is full or the source is empty.

Solution (shortest: 6 steps)

Step	Action	4-gal jug (x)	3-gal jug (y)
0	Start	0	0
1	Fill the 3-gallon jug from the pump	0	3
2	Pour the 3-gallon jug into the 4-gallon jug	3	0
3	Fill the 3-gallon jug again	3	3
4	Pour 3-gallon jug into 4-gallon jug until it is full (only 1 gallon fits)	4	2
5	Empty the 4-gallon jug onto the ground	0	2
6	Pour the 3-gallon jug (2 gallons) into the 4-gallon jug	2	0

Answer: Following the six steps above, the 4-gallon jug ends with exactly 2 gallons: state (2, 0). Note that after step 4 the 3-gallon jug already holds exactly 2 gallons, so steps 5–6 simply move them into the 4-gallon jug.

Alternative route (starting by filling the 4-gallon jug): (0,0) → (4,0) → (1,3) → (1,0) → (0,1) → (4,1) → (2,3). This also reaches 2 gallons (in the 3-gallon jug) after 6 moves; the first solution is preferred because it leaves the 2 gallons in the 4-gallon jug.

Question 2: Mars Rover – Analysing and Exploring Samples

Question: *Answer the percept, environment, action, performance and architecture questions for a Mars rover agent.*

i. Percepts of the agent

Percepts are what the rover senses at any instant through its sensors:

- Camera images (stereo / panoramic / microscopic) of terrain, rocks and soil.
- Spectrometer and chemical-sensor readings (mineral and chemical composition, presence of water or organics).
- Terrain and position data: GPS-like navigation, inertial unit (tilt, orientation), wheel encoders, obstacle and slope sensors.
- Environmental data: temperature, wind, dust, radiation and light levels.
- Internal state: battery level, solar-panel output, motor and instrument health.
- Commands and updates received from Earth (with communication delay).

ii. Operating environment

Property	Characterisation
Observability	Partially observable – sensors see only a limited area; subsurface and distant features are unknown.
Agents	Single agent (no competing agents; Earth operators act only as high-level supervisors).
Determinism	Stochastic – wheel slip, dust storms and uncertain soil make action outcomes unpredictable.
Episodic / Sequential	Sequential – current moves (e.g. getting stuck) affect all future actions and energy.
Static / Dynamic	Dynamic – weather, light and temperature change while the rover is deliberating.
Discrete / Continuous	Continuous – positions, speeds, temperatures and time are continuous values.
Known / Unknown	Largely unknown – terrain and samples are new, so the rover must learn about it.

iii. Actions the agent can take

- Move forward / backward, turn left / right, climb, stop, avoid obstacles.
- Capture images and take sensor readings; deploy the robotic arm.
- Scoop, drill or grind soil and rock; place samples in the onboard laboratory for analysis.
- Store, label or discard samples; clean instruments.
- Transmit data to Earth / orbiters, and receive commands.
- Manage power: recharge using solar panels, enter sleep or safe mode, heat or cool components.

iv. Performance measure

- Number and scientific value (quality) of samples collected and correctly analysed.
- Area of terrain explored and number of target sites reached.
- Safety and survival of the rover: no collisions, no falls, no getting stuck, instruments undamaged.
- Energy and time efficiency: science obtained per unit of power or time.
- Amount of useful data successfully transmitted to Earth.

v. Most suitable agent architecture

> **Answer:** A utility-based, learning agent (a goal-based agent with a utility function plus learning), built on a model-based reflex layer for safety.

Why:

- The environment is partially observable, so the agent needs an internal model of the world (map, battery and position) – a simple reflex agent is not enough.
- Radio delay of 4–24 minutes means Earth cannot control every move, so the rover must pursue goals autonomously (e.g. “reach that rock and analyse it”).
- Many goals compete (science value vs. battery vs. risk vs. time), so a utility function is needed to choose the best trade-off rather than just any goal-achieving action.
- The terrain is unknown, so a learning component helps it improve navigation and sample selection over time.
- Simple reflexes (stop at a cliff, avoid a boulder) are still kept for immediate safety.

Question 3: 8-Queens Problem

Question: *Explore the search space using search strategies and formulate the problem components for the 8-Queens problem (place 8 queens so none attacks another).*

Problem formulation

Component	Definition
States	Any arrangement of 0 to 8 queens on the board. (Efficient version: one queen per column, placed in the leftmost empty column, with no queen attacking another.)
Initial state	Empty 8×8 chessboard (no queens).
Actions	Add a queen to a square in the leftmost empty column such that it is not attacked by any queen already placed.
Transition model	Returns the board with the new queen added to the chosen square.
Goal test	8 queens are on the board and no two queens share the same row, column or diagonal.
Path cost	Not important – only the final arrangement matters (each placement may be given cost 1).

Size of the search space

Formulation	States	Notes
Place 8 queens anywhere (any squares)	$64C8 \approx 4.4 \times 10^9$	Very large
One queen per column, any row	$8^8 = 16{,}777{,}216$	Row conflicts still possible
One queen per row and column (permutations)	$8! = 40{,}320$	Only diagonals can conflict
Incremental, no attacked squares allowed	≈ 2,057 states	Best formulation

Search strategies

- **Depth-first search with backtracking:** place a queen column by column; if a column has no safe row, backtrack to the previous column and move that queen. Memory-efficient and the standard method for N-Queens.
- **Breadth-first search:** guaranteed to find a solution but stores a very large number of partial boards, so it is impractical.
- **Local search (hill climbing / min-conflicts):** start with 8 queens (one per column), count attacking pairs (the heuristic h) and move a queen within its column to reduce h; random restarts avoid local minima. Solves even very large N quickly.

Analysis of the configuration in the figure

The board in the question has all 8 queens placed, but the queen in the first column and the queen in the last column lie on the same diagonal. Two queens on one diagonal attack each other, so the goal test fails ($h \geq 1$) and this is not a solution. A search algorithm would reject it: DFS would backtrack and move one of those two queens to another row, and min-conflicts would move one of them to the row giving the fewest attacks.

A valid solution

Rows (1–8) of the queens in columns 1 to 8: 1, 5, 8, 6, 3, 7, 2, 4. No two share a row, column or diagonal.

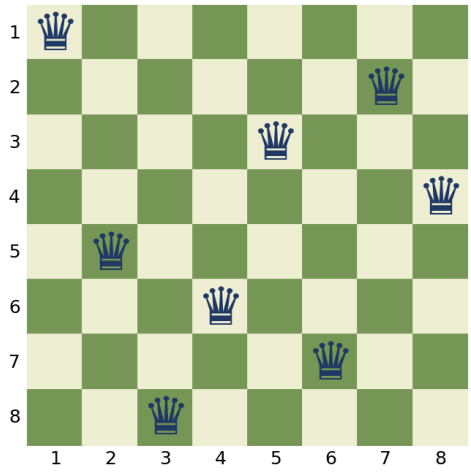


Figure 1: One valid 8-Queens solution

Answer: The problem is formulated as an incremental search (column by column, safe squares only); DFS with backtracking or min-conflicts local search finds a solution such as (1,5,8,6,3,7,2,4). The configuration in the figure is invalid because the first- and last-column queens attack each other diagonally.

Question 4: OLA Cab Booking Application

Question: *A customer selects a cab type (mini, micro, sedan, shared, prime, etc.) based on comfort to reach the destination. Identify the type of problem-solving agent and write pseudocode.*

Type of agent

> **Answer:** A utility-based agent (a goal-based problem-solving agent that also uses a utility function).

- **Goal:** reach the destination.
- **Why not purely goal-based:** every cab type reaches the goal, so the agent must compare them by utility – comfort, fare, waiting time, travel time, and availability – and choose the best one.
- **PEAS summary:** Percepts – pickup/drop locations, available cab types, fares, ETA, traffic. Actions – show options, book, confirm, cancel. Performance – customer satisfaction (comfort), cost, time. Environment – partially observable, dynamic and stochastic.

Pseudocode

```
FUNCTION BookCab(pickup, drop, user_preference):
    // 1. Formulate goal and problem
    goal      <- drop
    distance  <- GetRoute(pickup, drop)      // distance, traffic
    cabTypes  <- {Mini, Micro, Sedan, Shared, Prime}
    // 2. Search / collect available options
    options <- empty list
    FOR EACH cab IN cabTypes DO
        IF Available(cab, pickup) THEN
            fare <- CalculateFare(cab, distance)
            eta  <- EstimatedArrival(cab, pickup)
            ADD (cab, fare, eta) TO options
        END IF
    END FOR
    IF options is empty THEN
        RETURN "No cabs available - try again later"
    END IF
    // 3. Evaluate each option with the utility function
    bestCab <- NULL;  bestUtility <- -INFINITY
    FOR EACH (cab, fare, eta) IN options DO
        U <- w1*Comfort(cab) - w2*fare - w3*eta   // w = user weights
        IF U > bestUtility THEN
            bestUtility <- U
            bestCab     <- cab
        END IF
    END FOR
    // 4. Show options and let the customer choose
    DISPLAY options, MARK bestCab AS "Recommended"
    chosen <- CustomerSelection()            // customer may override
    // 5. Act: book the cab
    driver <- AssignDriver(chosen, pickup)
    CONFIRM booking(chosen, driver, fare)
    TrackRide(driver, drop)                  // until destination
    IF Reached(drop) THEN RETURN "Trip completed"
END FUNCTION
```

Question 5: Uniform Cost Search (UCS)

Question: *Find the least-cost path from the starting warehouse S to the destination warehouse G in the weighted delivery network using Uniform Cost Search.*

Graph as read from the figure

Edges: S–A 1, A–B 3, A–C 1, B–D 3, C–D 1, C–G 2, D–G 3, S–G 12.

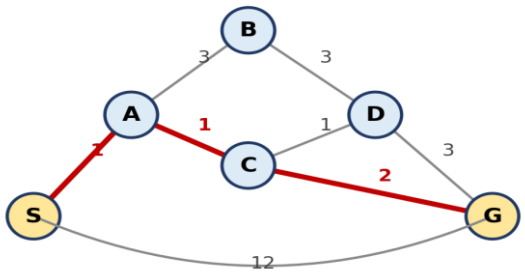


Figure 2: Weighted delivery network (least-cost path S → A → C → G highlighted)

How UCS works

UCS keeps a priority queue ordered by path cost g(n) and always expands the node with the lowest cumulative cost. It stops when the goal is removed from the queue (not when it is first generated), which guarantees an optimal path.

Step-by-step execution

Step	Node expanded (cost)	Priority queue after expansion (node : cost)	Remark
1	S (0)	A:1, G:12	Start node
2	A (1)	C:2, B:4, G:12	A reached at 1; B = 1+3, C = 1+1
3	C (2)	D:3, B:4, G:4	G improves from 12 to 4 via C (2+2)
4	D (3)	B:4, G:4	G via D = 6 > 4, rejected
5	B (4)	G:4	D via B = 7 > 3, rejected
6	G (4)	—	Goal removed from queue: stop

Path comparison

Path	Total cost	Result
S → A → C → G	1 + 1 + 2 = 4	Optimal
S → A → C → D → G	1 + 1 + 1 + 3 = 6	Higher
S → A → B → D → G	1 + 3 + 3 + 3 = 10	Higher
S → G (direct)	12	Highest

Answer: The least-cost path is S → A → C → G with a total cost of 4. Nodes were expanded in the order S, A, C, D, B, G.

Properties of UCS: complete (when all costs are positive) and optimal; time and space complexity are O(b^(1+⌊C*/ε⌋)), where C* is the optimal cost and ε the minimum edge cost.

Note: edge weights were read from the figure; if any differs in your paper, re-run the table the same way.